

Claude Vibe Coding & Engineering

A Complete Guide

5 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

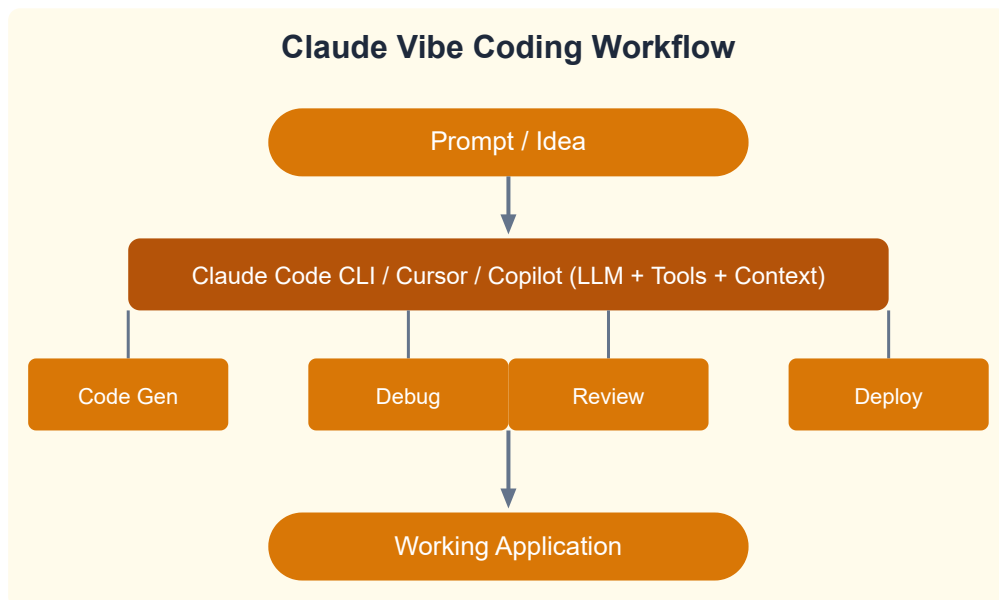
This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:
<https://snehaliteng.github.io/tutorials/claude-vibe-course/>

Vibe Coding for Fun & Profit

Core Concepts

"Vibe coding" is an AI-native coding paradigm where you describe what you want in natural language and let AI generate the code. The shift is from writing code line-by-line to curating AI-generated output.



AI Foundations

Concept	Description
LLMs	Large Language Models power code generation (Claude, GPT-4o)
Tools & Loops	AI reads files, runs commands, iterates on feedback
Context Engineering	Writing effective prompts and files <code>agents.md</code> / <code>claude.md</code>
Workflow Evolution	From manual coding to AI-assisted to fully AI-driven

IDE Hands-On

Compare the major AI coding IDEs:

Tool	Strengths	Best For
Cursor	Deep context awareness, inline editing	Daily development
Copilot	GitHub integration, VS Code native	Enterprise teams
Codex CLI	Terminal-native, agentic actions	Automation scripts
Claude Code	Full project context, checkpoints	Complex codebases

Building a Kanban App

```
# In Cursor or Claude Code CLI, you can prompt:
"Create a full-stack Kanban board with FastAPI backend and vanilla JS frontend.
Include: drag-and-drop, add/delete columns, persist to SQLite."

# The AI will generate:
# backend/main.py - FastAPI routes
# frontend/index.html - Drag-and-drop UI
# backend/models.py - SQLite models
```

Multi-Model Environments

- **YOLO Mode** — Let AI run commands autonomously
- **Responsible Coding** — Always review AI-generated code for security
- **Cross-Model Collaboration** — Switch between Claude, GPT-4o, and local models

Full-Stack Workflows

Build commercial MVPs quickly: FastAPI + Docker + SQLite + a modern frontend.

```
# FastAPI setup
pip install fastapi uvicorn sqlalchemy

# Dockerfile
FROM python:3.11
COPY . /app
RUN pip install -r requirements.txt
CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

 **Exercise:** Use Claude Code CLI or Cursor to build a Kanban board app. Start with "Create a Kanban board with FastAPI backend and HTML/CSS/JS frontend." Iterate by adding drag-and-drop, local persistence, and column management. Use Docker to containerize the final product.

Vibe Engineering as a Professional

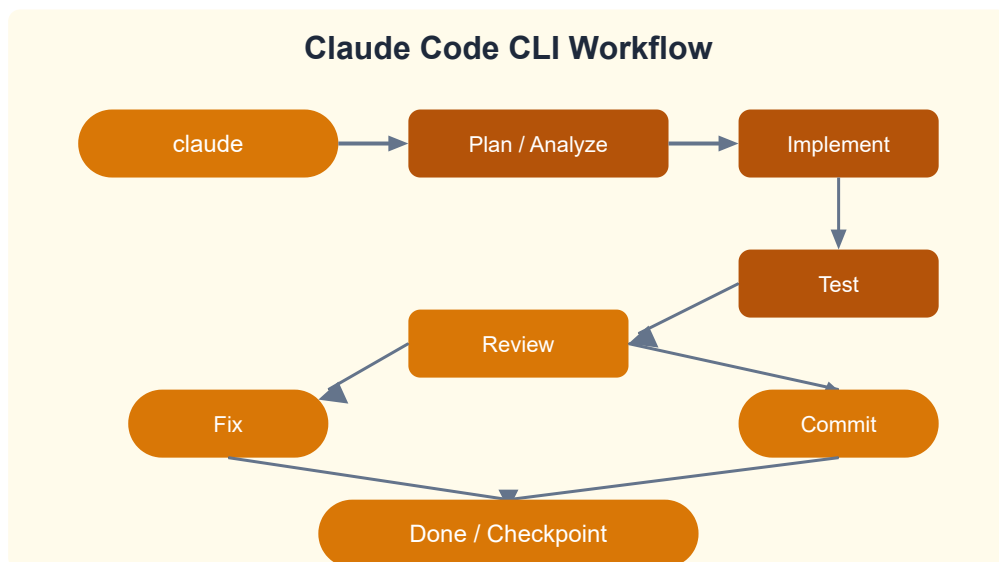
Claude Code CLI

Claude Code is Anthropic's agentic CLI tool. It operates in your terminal with full file system access, git integration, and the ability to run commands.

```
# Installation
npm install -g @anthropic-ai/claude-code

# Start a session
claude

# Or pipe a prompt directly
echo "Explain this codebase" | claude -p
```



Advanced Controls

Command	Description
<code>/plan</code>	Generate a plan before writing code
<code>/review</code>	Review recent changes
<code>/checkpoint</code>	Save a session checkpoint
<code>--yolo</code>	Autonomous mode — run commands without approval

```
# Using checkpoints
claude --checkpoint save "pre-refactor"
# ... make changes ...
claude --checkpoint load "pre-refactor" # Rollback if needed
```

Ecosystem Extensions: MCP Servers

Model Context Protocol (MCP) servers extend Claude's capabilities with custom tools.

```
// MCP server configuration in claude.json
{
  "mcpServers": {
    "jira": {
      "command": "node",
      "args": ["mcp-jira-server.js"],
      "env": { "JIRA_TOKEN": "${JIRA_TOKEN}" }
    },
    "github": {
      "command": "node",
      "args": ["mcp-github-server.js"]
    }
  }
}
```

Professional Integrations

- **Jira** — Create and update tickets from Claude
- **GitHub** — PR review, issue management, code review
- **Custom Skills** — Reusable prompt templates for common tasks

Production SaaS Project

Build a legal document generator SaaS with:

- `claude.md` project context file
- Custom skills for legal document generation
- Jira integration for feature tracking
- FastAPI backend with user authentication
- Automated deployment pipeline

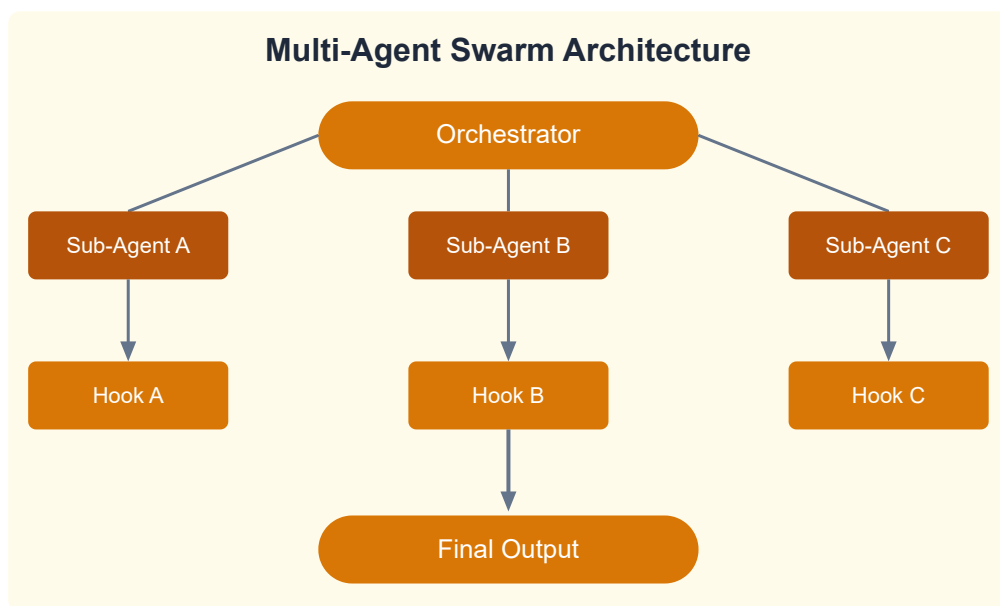
```
# claude.md – Project context file
# Project: LegalDoc Generator
# Stack: FastAPI, PostgreSQL, Jinja2 templates
# Conventions: Black formatting, pytest, async views
# Key files: /app/main.py, /app/models/, /app/templates/
```

 **Exercise:** Set up Claude Code CLI in a project. Configure an MCP server for GitHub (issue management). Create a `claude.md` context file. Build a small SaaS feature end-to-end: create a Jira ticket from Claude, implement the feature, open a PR, and review the PR — all from the CLI.

Vibe Engineering as an Expert

Multi-Agent Architectures

Expert-level vibe engineering uses swarms of sub-agents, each with specialized roles, orchestrated by hooks and custom commands.



Sub-Agents & Hooks

```
// Claude swarm configuration
{
  "agents": {
    "frontend": {
      "role": "Frontend Developer",
      "context": ["/frontend/**", "tailwind.config.js"],
      "hooks": {
        "pre-commit": "lint:check",
        "post-deploy": "e2e:test"
      }
    },
    "backend": {
      "role": "Backend Developer",
      "context": ["/backend/**", "requirements.txt"],
      "hooks": {
        "pre-commit": "pytest",
        "post-merge": "migrate:db"
      }
    }
  }
}
```

Controlled Chaos: Swarms

Swarms are groups of agents that work in parallel on different aspects of a problem. An orchestrator coordinates their outputs.

```
# Swarm orchestrator pattern
class SwarmOrchestrator:
    def __init__(self):
        self.agents = {
            "planner": Agent("plan"),
            "coder": Agent("code"),
            "tester": Agent("test"),
            "reviewer": Agent("review")
        }

    async def run(self, task):
        plan = await self.agents["planner"].run(task)
        code = await self.agents["coder"].run(plan)
        test_report = await self.agents["tester"].run(code)
        review = await self.agents["reviewer"].run(code, test_report)
        return review
```

Cloud Sandboxes

Execute agents remotely in cloud sandboxes (Sprites.dev, Modal) for secure, scalable execution.

```
claude --sandbox sprites # Run in cloud sandbox
```

Scale & SDKs

The Claude Agent SDK enables programmatic agent creation and orchestration for large codebases.

```
pip install claude-agent-sdk

from claude_agent import Agent, Swarm

agent = Agent(
    name="CodeReviewer",
    model="claude-sonnet-4-20250514",
    instructions="Review code for bugs, security, and style."
)

swarm = Swarm(agents=[agent], orchestrator="sequential")
result = swarm.run("Review the PR #42 changes")
```

Advanced Swarms & Capstone

The Gastown orchestration pattern enables parallel agent execution with dynamic task assignment.

Capstone: Trader Workstation

Build a complete trading platform using multi-agent swarms:

- **Data Agent** — Fetches market data from APIs
- **Analysis Agent** — Runs technical analysis
- **Risk Agent** — Assesses portfolio risk
- **Execution Agent** — Places trades via broker API
- **Reporting Agent** — Generates performance reports

 **Exercise:** Build a multi-agent swarm for a trading platform. Configure sub-agents for data, analysis, risk, and execution. Use hooks for pre-trade checks and post-trade logging. Deploy the orchestrator to a cloud sandbox. The final system should be able to: fetch real-time data, analyze trends, assess risk, and execute paper trades autonomously.

[← Back to Tutorials](#)

4. Practice & Resources

Quick Reference Guide

Task	Command / Tool
Start Claude Code CLI	<code>claude</code>
Pipe prompt to Claude	<code>echo "prompt" claude -p</code>
Save checkpoint	<code>claude --checkpoint save "name"</code>
Load checkpoint	<code>claude --checkpoint load "name"</code>
YOLO mode	<code>claude --yolo</code>
Plan before coding	<code>/plan</code> command in Claude
Review changes	<code>/review</code> command in Claude
MCP server config	<code>claude.json</code> → <code>mcpServers</code>
Cloud sandbox	<code>claude --sandbox sprites</code>
Project context	<code>claude.md</code> file in project root

Useful Resources

- [Claude Code CLI Documentation](#)
- [MCP Specification](#)
- [Cursor IDE](#)
- [GitHub Copilot](#)
- [Sprites.dev — Cloud sandbox for AI coding](#)

3-Week Study Plan

Week	Topic	Activities
1	Vibe Coding	Setup Cursor/Claude Code, build Kanban app, learn context engineering
2	Vibe Engineering	Claude Code CLI mastery, MCP servers, Jira/GitHub integration, SaaS project
3	Expert Engineering	Multi-agent swarms, hooks, cloud sandboxes, trading platform capstone

IDE / Tool Comparison

Tool	Cost	Best For	AI Models
Cursor	\$20/mo	Daily coding with deep context	Claude, GPT-4o
Copilot	\$10/mo	Enterprise teams, VS Code	GPT-4o, Claude
Claude Code CLI	API usage	Complex projects, agentic workflows	Claude
Codex CLI	API usage	Terminal-native automation	GPT-4o
OpenCode	Open source	Self-hosted, custom workflows	Various

 **Final Capstone:** Build a complete multi-agent SaaS platform using all three levels: (1) vibe-code the initial prototype, (2) engineer it professionally with MCP servers, claude.md context, and CI/CD, (3) add expert-level multi-agent swarm orchestration for critical workflows. Deploy with cloud sandboxes and document the entire process.

5. References

Glossary

Term	Definition
Vibe Coding	AI-native coding where you describe intent and AI generates code
Claude Code CLI	Anthropic's terminal-based agentic coding tool
MCP	Model Context Protocol — standardized tool interface for AI
Context File	<code>claude.md</code> or <code>agents.md</code> — project instructions for AI
Checkpoint	Saved session state for rollback
YOLO Mode	Autonomous mode where AI runs commands without approval
Sub-Agent	Specialized agent within a larger swarm
Hook	Automated action triggered by events (pre-commit, post-deploy)
Swarm	Group of coordinated agents working in parallel
Sandbox	Isolated cloud environment for safe AI code execution

Cheatsheet

```
# Claude Code CLI Cheatsheet

# Install
npm install -g @anthropic-ai/claude-code

# Start session
claude

# Quick prompt
echo "explain this" | claude -p

# Session controls
/plan      # Generate a plan
/review    # Review changes
/summarize # Get session summary
/checkpoint # Save state

# Flags
--yolo      # Autonomous mode
--model MODEL # Use specific Claude model
--resume    # Resume last session
--verbose   # Detailed output

# Config file (claude.json)
{
  "mcpServers": { "serverName": { "command": "node", "args": ["server.js"] } }
}

# Project context (claude.md)
# Place in project root with conventions, stack, and key files.
```

Claude Model Comparison

Model	Best For	Context
Claude Haiku	Fast, lightweight tasks	200K
Claude Sonnet	Balanced speed/quality	200K
Claude Opus	Complex reasoning, code	200K

 **Congratulations!** You've completed the Claude Vibe Coding & Engineering course. You can now vibe-code prototypes, engineer production systems with Claude Code CLI, and orchestrate expert-level multi-agent swarms for complex applications.